

Unsere ersten Anweisungen

- Kommunikation mit dem Benutzer über die Konsole
 - User Interface

- Ausgabe

```
System.Console.WriteLine(text);
```

- Gibt den Text auf den Bildschirm aus und positioniert Cursor an den Beginn der nächsten Zeile (Zeilenvorschub)

```
System.Console.Write(text);
```

- Gibt den Text auf den Bildschirm aus und positioniert Cursor an das Ende der ausgegebenen Zeile

- Eingabe von Texten

```
string eingabe;
```

```
eingabe = System.Console.ReadLine();
```

- Liest eine Zeile (bis zum Drücken der Enter-Taste) ein und speichert den Text in der Stringvariablen `eingabe`

Beispiel: Eine Zeichenkette einlesen

```
public static void Main()
{
    Console.WriteLine("Hey my friend, what's your name?");
    string name = Console.ReadLine();
    Console.WriteLine("Nice to see you, " + name + "!");

    Console.WriteLine();
    Console.WriteLine("Press any key to exit the program ...");
    Console.ReadKey();
}
```



```
Hey my friend, what's your name?
Birgit
Nice to see you, Birgit!

Press any key to exit the program ...
```

Deklaration und Zuweisung

Deklaration:
Die Variable heißt „name“
und ist vom Datentyp
„string“ (Zeichenkette)

Zuweisung: Die in der
Konsole eingegebene
Zeichenkette wird der
Variablen zugewiesen

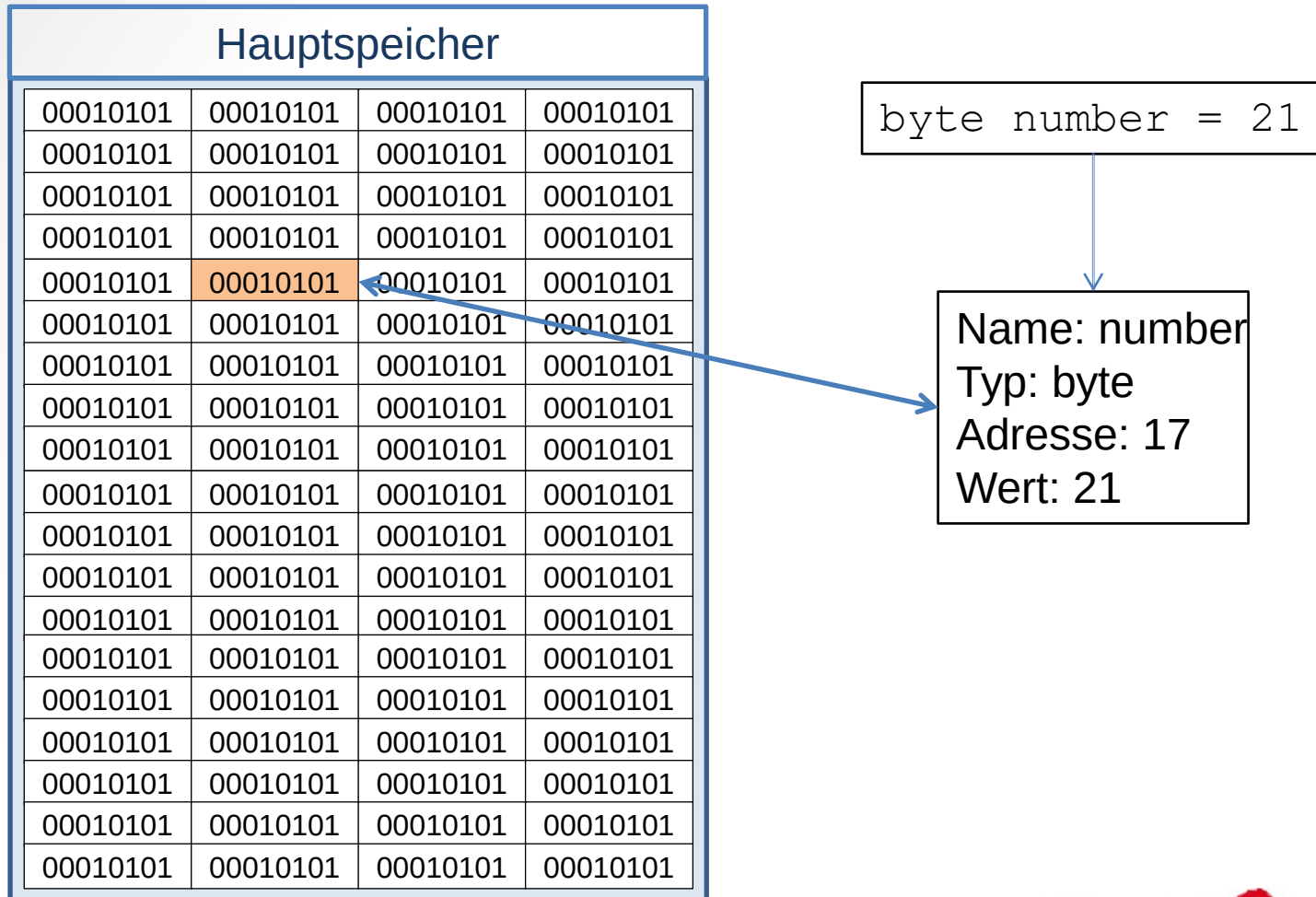
```
string name = Console.ReadLine();
```

- Man kann ab jetzt der Variablen auch andere Zeichenketten zuweisen:

```
name = "Rumpelstilzchen";
```

Variablen

Daten, die im Programm mehrmals benötigt werden, können als Variablen gespeichert werden.



Datentyp

Für den Prozessor sind Daten eine Folge von zwei verschiedenen Zuständen.

0

Bit (Binary Digit), 2 Zustände

0	1	1	0	0	1	0	0
---	---	---	---	---	---	---	---

Byte, $2^8 = 256$ Zustände

In einer Programmiersprache werden ein oder mehrere Bytes zu sogenannten Datentypen zusammengefasst.

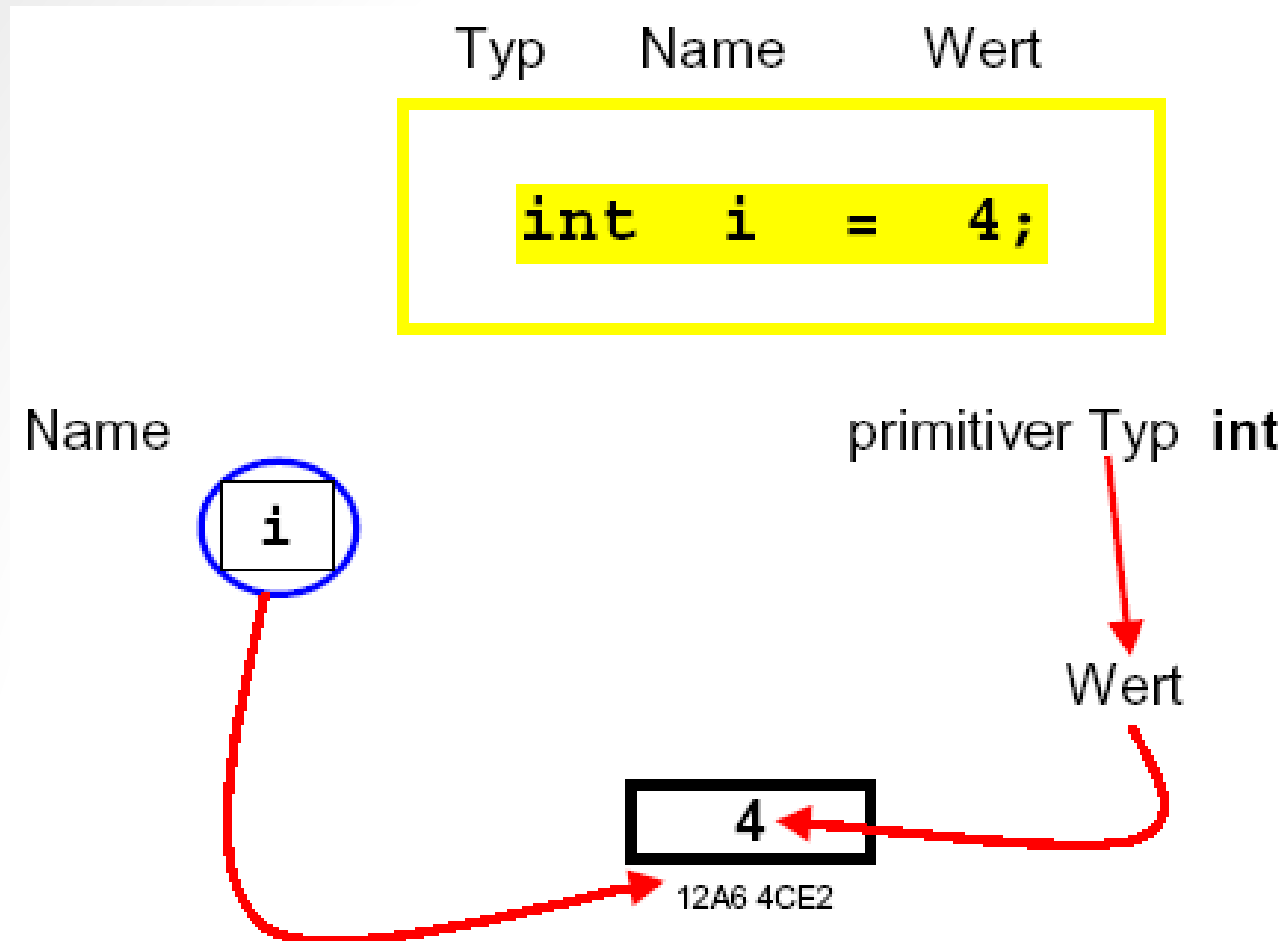
0	1	1	0	0	1	0	0	0	1	1	0	0	1	0	0	0	1	1	0	0	1	0	0	0	1	1	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

z. Bsp. `int` (4 Byte), $2^{32} = 4.294.967.296$ Zustände

Variablen: Name, Datentyp, Daten

- Variablen sind **benannte Platzhalter** bestimmter Größe für Bereiche im Arbeitsspeicher, in denen Daten gespeichert sind:
 - Welchen **Namen** hat die Variable? (**Deklaration** → Bezeichner)
 - Welchen **Datentyp** hat die Variable? (**Definition** → Größe im Speicher)
 - Welche **Daten** werden gespeichert? (**Initialisierung** → Wertzuweisung)
- Deklaration und Definition:
 - Typ Bezeichner; `z.B. int counter;`
- Initialisierung / Zuweisung:
 - Bezeichner = Wert; `z.B. counter = 4;`
- Zusammenfallen von Deklaration und Initialisierung:
 - Typ Bezeichner = Wert; `z.B. int counter = 4;`
- Alle Variablen müssen vor der Benutzung initialisiert werden (aber nicht zwangsläufig sofort bei der Deklaration)

Deklaration, Definition und Initialisierung von Variablen



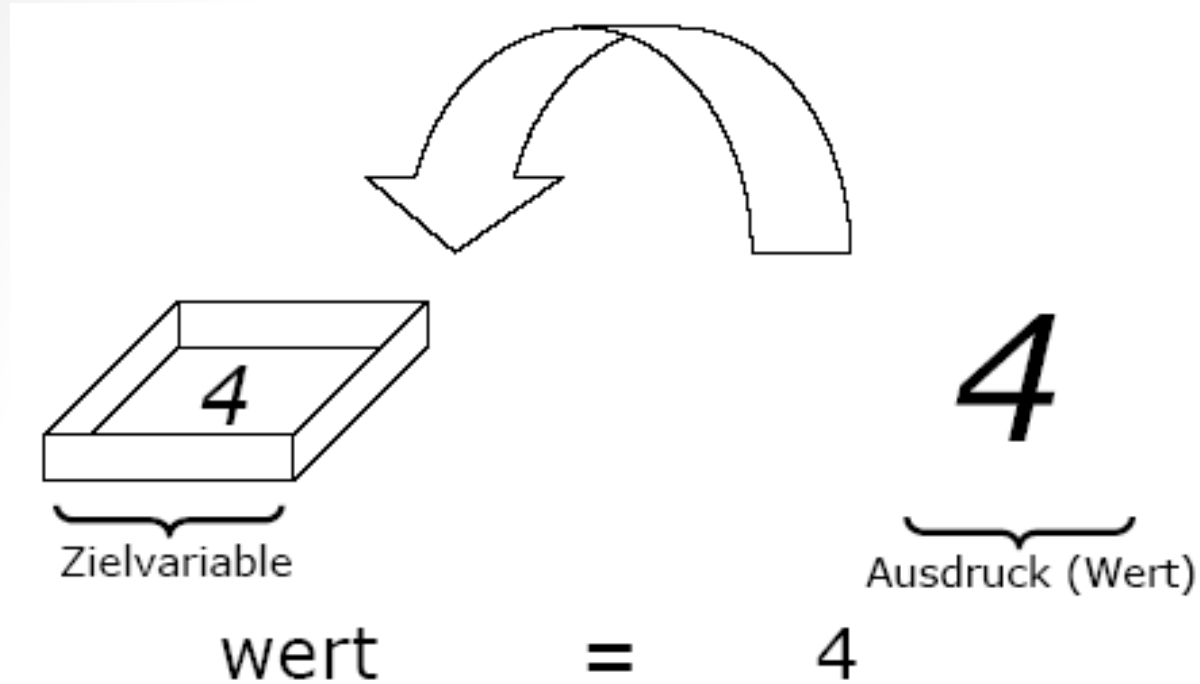
Wichtige Typen am Beginn der Karriere

- `string` Texte aus einem oder mehreren Zeichen
- `char` einzelnes Zeichen
- `int` Ganzzahl zwischen
 -2.147.483.648 und
 2.147.483.647
- `bool` Wahrheitswert `true` und `false`
- `double` Gleitkommazahl

Einfache Datentypen in C#

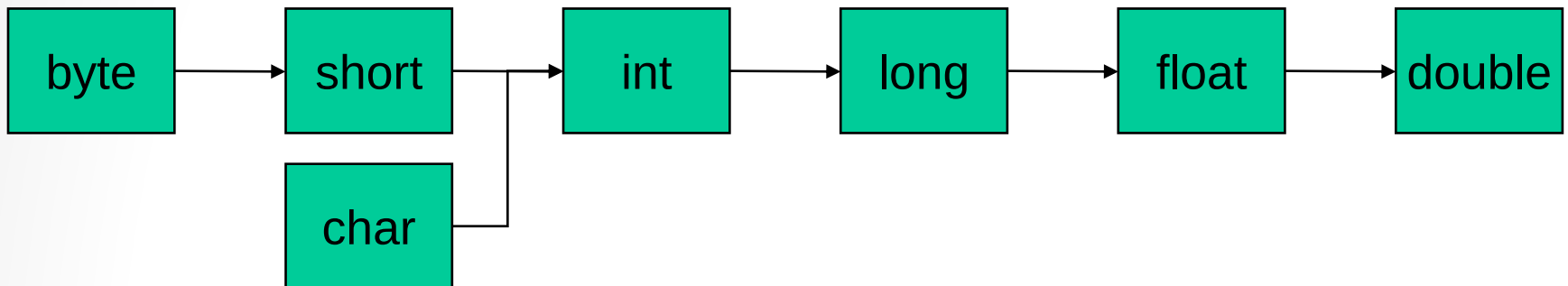
Schlüsselwort	abgebildet auf	Wertebereich
sbyte	System.SByte	-128 .. 127
short	System.Int16	-32768 .. 32767
int	System.Int32	-2 147 483 648 .. 2 147 483 647
long	System.Int64	$-2^{63} .. 2^{63}-1$
byte	System.Byte	0 .. 255
ushort	System.UInt16	0 .. 65535
uint	System.UInt32	0 .. 4294967295
ulong	System.UInt64	$0 .. 2^{64}-1$
float	System.Single	$\pm 1.5E-45 .. \pm 3.4E38$ (32 Bit)
double	System.Double	$\pm 5E-324 .. \pm 1.7E308$ (64 Bit)
decimal	System.Decimal	$\pm 1E-28 .. \pm 7.9E28$ (128 Bit)
bool	System.Boolean	true, false
char	System.Char	Unicode-Zeichen

Zuweisungsoperator



Zuweisungsoperator

- Vorsicht
 - Zugewiesener Wert muss zum Typ der Variablen kompatibel sein (im verfügbaren Speicher Platz haben)
 - Klein in Groß funktioniert, aber nicht umgekehrt!
 - Zuweisung kann aber erzwungen werden



Bezeichner – Namen von Variablen

- Kombination aus Zeichen, Ziffern, _, @
- Case-sensitiv
- Schlüsselwörter dürfen nicht verwendet werden
 - Siehe <https://msdn.microsoft.com/de-de/library/x53a06bb%28v=vs.120%29.aspx>
- Richtlinien für die Namenswahl:
 - Sprechende Bezeichner verwenden – der Name beschreibt, was in der Variablen gespeichert ist!
 - Nur englische Namen verwenden
 - „CamelCase“
 - Beginnen mit kleinem Anfangsbuchstaben
 - Jedes innere Teilwort groß beginnen
 - Bsp: *myVariableInCamelCaseNotation*



Variablen - Beispiele

- `string someText = "Important text";`
 - `int value = 42;`
 - `bool isValid = true;`
 - `double decimalValue = 3.141592653;`
- Punkt oder Komma?
- `string firstName = "Hansi";`
 - `string lastName = "Hinterseer";`
 - `Console.WriteLine(firstName + " " + lastName);`
-
- `int summand1 = 17;`
 - `int summand2 = 42;`
 - `int sum = summand1 + summand2;`
 - `Console.WriteLine("Die Summe beträgt " + sum);`

Einlesen von Zahlen

- `string userInput;`
- `userInput = Console.ReadLine();`
- `int value = Convert.ToInt32(userInput);`
- `double decimalValue = Convert.ToDouble(userInput);`

Frage: Wie liest man eine Zahl ein?

Bsp: Einlesen eines Gleitkomma-Bruches

```
public static void Main()
{
    Console.WriteLine("Fraction to Decimal!");

    Console.Write("Enter numerator: ");
    string userInput = Console.ReadLine();
    double numerator = Convert.ToDouble(userInput);

    Console.Write("Enter denominator: ");
    userInput = Console.ReadLine();
    double denominator = Convert.ToDouble(userInput);

    double decimalValue = numerator / denominator;
    Console.WriteLine("{0}/{1} = {2}",
        numerator, denominator, decimalValue );

    Console.WriteLine();
    Console.WriteLine("Press any key to exit the program ...");
    Console.ReadKey();
}
```

Ausgabe von Variablen mit Platzhaltern

- `Console.Write("Hello {0}", name);` // Platzhalter
- `Console.WriteLine("{0} = {1}", x, y);`

Platzhalter-Syntax

```
"{" n ["," width] [":" format [precision]] "}"
```

- n* Argumentnummer (beginnend bei 0)
- width* Feldbreite (wenn zu klein, wird sie überschritten)
 positiv = rechtsbündig, negativ = linksbündig
- format* Formatierungscode (z.B. d, f, e, x, ...)
- precision* Anzahl der Nachkommastellen (machmal Anzahl der Ziffern)

Beispiel: `{0,10:f2}`

Seit C# 6 noch besser: \$

- `Console.Write($"Hello {name}");` // Platzhalter für Variable
- `Console.WriteLine($"{x} = {y}");`

Platzhalter-Syntax

```
"{" variable ["," width] [":" format [precision]] "}"
```

variable Variablenname

width Feldbreite (wenn zu klein, wird sie überschritten)
 positiv = rechtsbündig, negativ = linksbündig

format Formatierungscode (z.B. d, f, e, x, ...)

precision Anzahl der Nachkommastellen (machmal Anzahl der Ziffern)

Beispiel: `{floatingPointNumber,10:f2}`

Formatierungscode für Zahlen

d, D	Dezimalformat (ganze Zahl mit führenden Nullen) precision = Anz. Ziffern	-xxxxx
f, F	Fixpunktformat precision = Anz. Nachkommastellen (Default = 2)	-xxxxx.xx
n, N	Nummernformat (mit Tausender-Trennstrich) precision = Anz. Nachkommastellen (Default = 2)	-xx,xxx.xx
e, E	Exponentialformat (groß/klein signifikant) precision = Anz. Nachkommastellen	-x.xxxE+xxx
c, C	Currency-Format precision = Anz. Nachkommastellen (Default = 2) Negative Werte werden in Klammern gesetzt	\$xx,xxx.xx (\$xx,xxx.xx)
x, X	Hexadezimalformat (groß/klein signifikant) precision = Anzahl Hex-Ziffern (evtl. führende 0)	xxx
g, G	General (kompaktestes Format für gegebenen Wert; Default)	

Zahlenformatierung: Beispiele

```
int x = 17;
```

```
Console.WriteLine($"{x}");
```

17

```
Console.WriteLine($"{x,5}");
```

17

```
Console.WriteLine($"{x:d}");
```

17

```
Console.WriteLine($"{x,5:d3}");
```

017

```
Console.WriteLine($"{x:f}");
```

17.00

```
Console.WriteLine($"{x:f1}");
```

17.0

```
Console.WriteLine($"{x:E}");
```

1.700000E+001

```
Console.WriteLine($"{x:e1}");
```

1.7e+001

```
Console.WriteLine($"{x:x}");
```

11

```
Console.WriteLine($"{x:x4}");
```

0011

Konstanten

- Schlüsselwort: **const**
- Verwendung:
 - Fixe Werte, die sich während des Programmablaufs nie ändern, und im Programm mehrmals benötigt werden.
- Richtlinien:
 - Immer groß schreiben
 - Am Beginn des Programmcodes sammeln
- Beispiele:

```
const int GROWN_UP_AGE = 19;
```

```
const float TAX_RATE = 20.0;
```

Kommentare

```
// Dieser Teil wird vom Compiler ignoriert  
/* Da können wir irgendetwas schreiben */  
int x = 0;    // aber es sollte schon ein sinnvoller Kommentar sein  
/* vor */ x = 5; /* und nach einer "richtigen" Anweisung */  
/* wieder vor */ x = 10; // und jetzt wäre x = 11 völlig fehl am Platz ...
```

Am Besten das Programm so schreiben, dass man es auch ohne Kommentare leicht verstehen kann!

https://de.wikipedia.org/wiki/Clean_Code